

Towfish Magnetometer Software System

1 Overview

The software is split between a Raspberry Pi Zero controller and a Raspberry Pi Pico measurement processor. The Pi Zero owns the high-level measurement cycle: it controls the L1 and L2 GPIO outputs, polls the IMU, requests a frequency measurement from the Pico, corrects the magnetic-field result, and writes the final row to a CSV log. In this system, L1 represents the polarization circuit and L2 represents the sensing circuit. The Pico owns the timing-sensitive frequency measurement and GPS parsing. The Pico returns one response containing latitude, longitude, and measured frequency.

There are two operating pairs:

Mode	Pi Zero file	Pico file
Standard survey	SP26_HookLineandTowfish_pi_zero_controller_MattSchutt.py	SP26_HookLineandTowfish_pico_measure_uart_MattSchutt.py
Boat/charging integration	SP26_HookLineandTowfish_pi_zero_controller_boat_MattSchutt.py	SP26_HookLineandTowfish_pico_measure_uart_charge_MattSchutt.py

2 General Usage and Setup Notes

The current software is written in Python and MicroPython. If future users want to continue using Python for this system, the workflow below is the intended setup path.

2.1 Pi Pico Workflow

For the Pi Pico, Thonny is recommended. Thonny has a built-in MicroPython workflow and serial interface, so it usually requires less setup than VS Code for loading scripts, opening the REPL, and checking printed output from the Pico. VS Code can still be used, but it typically needs more extension and serial-device configuration before it is as convenient.

The Pico files are:

```
SP26_HookLineandTowfish_pico_measure_uart_MattSchutt.py
SP26_HookLineandTowfish_pico_measure_uart_charge_MattSchutt.py
```

Load the standard file for normal operation, or the charging-aware file if the boat integration is being used.

This is probably already familiar, but if a Pico program should start by itself without manually running it from Thonny or another IDE, save that program on the Pico as

```
main.py.
```

2.2 Pi Zero Workflow

For the Pi Zero, start by flashing Raspberry Pi OS using Raspberry Pi Imager. During imaging, enable SSH so the board can be reached without needing a monitor and keyboard. The current Pi Zero may still contain previous user information, so it should be reflashed and reconfigured before another user depends on it.

The first connection should be made over SSH. After logging in, open the Raspberry Pi configuration settings and update the peripheral interfaces:

1. Disable serial login. Leaving serial login enabled can interfere with the UART channels used by the Pico and boat communication.
2. Enable I2C for the MPU-6050 IMU.
3. Enable SPI if future hardware or debugging tools require it.
4. Keep the serial hardware interface enabled if UART communication with the Pico is being used, but keep serial console/login disabled.

If the user wants to work on the Pi Zero over SSH while at school, the easiest network setup is often to reflash or reconfigure the Pi Zero with the user's phone hotspot as the saved Wi-Fi network. This lets the user connect the laptop and Pi Zero to the same hotspot and SSH into the board without depending on a school network that may block device-to-device connections.

Similarly, if the Pi Zero should start the controller without manually launching it from an editor or terminal each time, place the startup entry in the Pi's normal boot path. A simple project convention is to make the script that should run the system the active

`main.py`

entry point, then configure the Pi's boot service or startup command to run that file.

3 Standard Pi Zero Controller

The standard controller is implemented in `SP26_HookLineandTowfish_pi_zero_controller_MattSchutt.py`. It runs continuously until interrupted. Each cycle performs the following sequence:

1. Turn L1 on for 1.6 s to energize the polarization circuit.
2. Turn L1 off.
3. Wait for `GAP_AFTER_L1_MS`. In the current code this is 0 ms.
4. Turn L2 on to enable the sensing circuit.
5. Send a UART command to the Pico:

`MEASURE <duration_ms>`

where the current duration is 1600 ms.

6. Poll the IMU for the full L2 measurement window.
7. Turn L2 off.

8. Wait for the Pico to return a line beginning with `DATA`.
9. Apply the IMU differential-orientation correction to the measured frequency.
10. Convert the corrected frequency to magnetic field in nanotesla.
11. Append latitude, longitude, and magnetic field to `magnetometer_log.csv`.
12. Wait 2s before starting the next cycle.

3.1 GPIO Outputs

The Pi Zero uses BCM GPIO numbering. The current pin assignments are

$$\text{L1_PIN} = 17, \quad \text{L2_PIN} = 27.$$

L1 drives the polarization circuit. L2 drives the sensing circuit used during the actual frequency measurement window.

These are wrapped by the `LedOutput` class. On the Raspberry Pi, the class calls `RPi.GPIO`. If the code is run on a non-Pi development machine, GPIO calls are printed instead of executed.

3.2 IMU Reader

The IMU is a GY-521 / MPU-6050 connected over I2C. The current configuration is

$$\text{I2C_BUS} = 1, \quad \text{MPU6050_ADDR} = 0x68.$$

During setup, the Pi Zero wakes the MPU-6050, configures the digital low-pass filter, sets a 100Hz sample rate, selects the $\pm 2g$ accelerometer range, and selects the $\pm 250^\circ \text{s}^{-1}$ gyro range. Each IMU sample contains:

$$a_x, a_y, a_z, g_x, g_y, g_z, T.$$

The acceleration values are scaled by 16384 counts per g , and the gyroscope values are scaled by 131 counts per degree per second. The present field correction uses the accelerometer-derived roll and pitch. The gyro values are read and retained in the sample dictionary, but they are not currently used in the correction calculation.

4 Pico Measurement Firmware

The standard Pico firmware is implemented in `SP26_HookLineandTowfish_pico_measure_uart_MattSchutt.py`. It has three main responsibilities:

1. Blink the onboard LED to confirm that the Pico is being powered by the Pi Zero, which is useful for debugging when both boards share the same power source.
2. Measure input frequency using the Pico PIO state machines.
3. Read GPS NMEA sentences and keep the most recent valid latitude and longitude.

4.1 UART Connections

The Pico uses two UARTs:

UART	Purpose	Pins
UART0	Pi Zero command/response link	GP0 TX, GP1 RX
UART1	GPS receiver input	GP4 TX, GP5 RX

UART0 runs at 9600 baud. The Pi Zero sends measurement commands on this link, and the Pico sends results back on the same link.

4.2 PIO Frequency Measurement

The Pico uses three PIO state machines:

State machine	PIO program	Role
SM0	<code>gate</code>	Creates the measurement gate interval
SM1	<code>clock_count</code>	Counts Pico clock ticks during the gate
SM2	<code>pulse_count</code>	Counts input pulses during the gate

The input signal is connected to GP15. The gate and pulse-finished helper pins are GP14 and GP13. For a requested measurement duration t_m , the gate count is

$$G = f_{\text{PIO}} \frac{t_m}{1000},$$

where

$$f_{\text{PIO}} = 125\,000\,000 \text{ Hz.}$$

After the PIO programs finish, the firmware computes:

$$C = 2 \left(2^{32} - 1 - C_{\text{raw}} + 1 \right),$$

$$P = 2^{32} - 1 - P_{\text{raw}},$$

where C is the measured clock-tick count and P is the measured input-pulse count. The measured frequency is then

$$f_{\text{meas}} = P \frac{f_{\text{PIO}}}{C}.$$

If the PIO measurement does not finish before its timeout, the Pico returns

$$f_{\text{meas}} = -1.0.$$

4.3 GPS Parsing

The Pico continuously drains the GPS UART in short bursts. It parses valid \$GPRMC or \$GNRMC sentences whose status field is A. Latitude and longitude are converted from NMEA degrees-minutes format into signed decimal degrees.

The Pico stores the most recent valid fix as

`last_lat,` `last_lon.`

Before a GPS fix is available, both values default to

0.000000, 0.000000.

5 Standard Pi Zero/Pico Protocol

The standard measurement command from the Pi Zero to the Pico is

```
MEASURE <duration_ms>
```

For the current controller settings, the transmitted command is

```
MEASURE 1600.
```

The Pico responds with

```
DATA <lat> <lon> <freq_hz>.
```

For example:

```
DATA 40.123456 -88.123456 2120.123456789012.
```

The Pi Zero accepts only lines beginning with DATA. If no valid line arrives before `UART_TIMEOUT_S`, currently 5 s, the controller raises a timeout error.

6 IMU Differential Orientation Correction

The Pico returns one frequency value integrated over the full measurement window. Because of this, the Pi Zero samples the IMU throughout the same window and uses the whole set of orientation differentials to compute one average correction factor.

This correction should be treated as provisional. The paper originally used as the reference for including IMU-based correction was later taken down, so the need for this correction in the final system is uncertain. At the time of this document, that leaves the correction roughly a 50/50 design decision: it is implemented and available, but it may be removed or disabled if testing shows that it does not improve the field measurements.

Other sources still mention orientation-related effects more loosely. For example, the ScienceDirect topic page on proton precession discusses sensor orientation effects such as heading errors, mechanical rotation effects, and orientation-dependent signal behavior in proton precession magnetometers:

<https://www.sciencedirect.com/topics/earth-and-planetary-sciences/proton-precession>.

This supports keeping orientation in mind, but it does not by itself prove that the IMU correction implemented here is required for this specific towfish design. Before making the final decision to include, disable, or remove this correction, the reader should do additional research to see whether newer papers, manufacturer notes, or application reports have clarified the usefulness of IMU orientation correction for this type of proton precession magnetometer.

6.1 Roll and Pitch From Accelerometer

For each IMU sample i , the accelerometer readings are

$$a_{x,i}, \quad a_{y,i}, \quad a_{z,i}.$$

The roll and pitch estimates are

$$\phi_i = \tan^{-1} \left(\frac{a_{y,i}}{a_{z,i}} \right),$$

$$\theta_i = \tan^{-1} \left(\frac{-a_{x,i}}{\sqrt{a_{y,i}^2 + a_{z,i}^2}} \right),$$

where ϕ_i is roll and θ_i is pitch.

6.2 Starting Orientation Baseline

The first N_b valid IMU samples define the starting orientation. In the controller code,

$$N_b = \min(5, N),$$

where N is the number of valid IMU samples collected during the measurement.

The baseline roll and pitch are

$$\begin{aligned} \phi_0 &= \frac{1}{N_b} \sum_{i=1}^{N_b} \phi_i, \\ \theta_0 &= \frac{1}{N_b} \sum_{i=1}^{N_b} \theta_i. \end{aligned}$$

6.3 Orientation Differentials

For each sample, the change in orientation relative to the measurement start is

$$\Delta\phi_i = \text{wrap}_{180}(\phi_i - \phi_0),$$

$$\Delta\theta_i = \text{wrap}_{180}(\theta_i - \theta_0).$$

The wrap function keeps angle differences in the interval $[-180^\circ, 180^\circ)$:

$$\text{wrap}_{180}(\alpha) = ((\alpha + 180^\circ) \bmod 360^\circ) - 180^\circ.$$

If either differential exceeds the configured limit,

$$|\Delta\phi_i| > 45^\circ \quad \text{or} \quad |\Delta\theta_i| > 45^\circ,$$

the controller rejects the IMU correction for that measurement and uses the raw frequency.

6.4 Projection Factor

For a single-axis, Z-oriented magnetic sensor, the measured component changes with roll and pitch according to

$$p_i = \cos(\Delta\theta_i) \cos(\Delta\phi_i).$$

The correction uses the average projection factor over the full measurement window:

$$\bar{p} = \frac{1}{N} \sum_{i=1}^N p_i.$$

If $\bar{p} \leq 0$ or is too close to zero, the controller rejects the IMU correction and uses the raw frequency.

6.5 Frequency and Field

Because magnetic field is proportional to measured precession frequency, the orientation correction is applied directly to frequency:

$$f_{\text{corr}} = \frac{f_{\text{meas}}}{\bar{p}}.$$

The corrected frequency is converted to magnetic field using the proton gyromagnetic ratio γ_p :

$$B = \frac{2\pi f_{\text{corr}}}{\gamma_p}.$$

In nanotesla,

$$B_{\text{nT}} = \frac{2\pi f_{\text{corr}}}{\gamma_p} \times 10^9.$$

The controller uses

$$\gamma_p = 2.67522 \times 10^8 \text{ rad s}^{-1} \text{ T}^{-1}.$$

7 CSV Logging

The Pi Zero writes data to `magnetometer_log.csv`. If the file does not exist, the controller creates it with the header

```
lat, lon, magnetic_field.nt.
```

Each successful measurement cycle appends one row:

```
<lat>, <lon>, <corrected field in nT>.
```

The current log does not include raw frequency, corrected frequency, IMU differentials, or correction status. Those values are printed to the console, but not written to the CSV.

8 Boat Integration Pi Zero Controller

The boat-aware controller is implemented in `SP26_HookLineandTowfish_pi_zero_controller_boat_MattSchutt.py`. It imports the core hardware, measurement, correction, conversion, and CSV helpers from `SP26_HookLineandTowfish_pi_zero_controller_MattSchutt.py`. This means the magnetic measurement and IMU correction are intentionally shared between the standard and boat modes.

The boat controller adds a second UART link to the boat software:

```
BOAT_UART_PORT = /dev/ttyAMA1,    BOAT_UART_BAUD = 9600.
```

8.1 Boat Commands

The boat controller listens for charging and run commands:

Command class	Accepted text
Charging	charging , charge , idle_charge
Run/resume	run , resume , measuring , measure

Commands are decoded as lowercase, newline-terminated text.

8.2 GO Message

Once the Pico has produced a nonzero GPS fix, the Pi Zero sends

GO

to the boat. The controller latches this state so it sends GO only once. A fix of 0,0 is treated as invalid because that is the Pico's startup placeholder before a real GPS fix arrives.

8.3 Charging Mode

When the boat sends a charging command, the Pi Zero stops the magnetometer measurement cycle. In this mode:

1. L1 and L2 are held low, so both the polarization and sensing circuits remain inactive.
2. No MEASURE command is sent to the Pico.
3. The Pi Zero periodically sends GPS_ONLY to the Pico.
4. GPS is polled every 10s.
5. The latest GPS fix can still trigger the one-time GO message if it has not already been sent.

This avoids energizing the measurement hardware while the boat is parked in its charging state.

8.4 Responsive Sleeps

The boat controller replaces long blocking sleeps with `sleep_with_boat_checks`. This function sleeps in short chunks and checks the boat UART between chunks. The result is that a charging command can interrupt L1 timing, the L1/L2 gap, or the post-cycle wait without waiting for a full long sleep to finish.

9 Charging-Aware Pico Firmware

The charging-aware Pico firmware is implemented in `SP26_HookLineandTowfish_pico_measure_uart_charge_MattSchutt.py`. It keeps the same PIO measurement logic, GPS parser, and standard MEASURE command behavior as `SP26_HookLineandTowfish_pico_measure_uart_MattSchutt.py`. The added command is

GPS_ONLY.

When the Pico receives GPS_ONLY, it drains the GPS UART for a longer interval, currently 250ms, and returns

GPS <lat> <lon>.

No PIO frequency measurement is started, and the measurement hardware is not used. This command exists specifically for the boat's idle charging mode.

10 Standard Versus Boat Mode Summary

Feature	Standard mode	Boat/charging mode
Pi Zero file	SP26_HookLineandTowfish_pi_zero_controller_MattSchutt.py	SP26_HookLineandTowfish_pi_zero_controller_boat_MattSchutt.py
Pico file	SP26_HookLineandTowfish_pico_measure_uart_MattSchutt.py	SP26_HookLineandTowfish_pico_measure_uart_charge_MattSchutt.py
Measurement command	MEASURE <ms>	MEASURE <ms>
Pico data response	DATA <lat> <lon> <freq>	DATA <lat> <lon> <freq>
IMU correction	Yes	Yes, imported from standard controller
Boat UART	No	Yes
One-time boat GO	No	Yes, after valid GPS fix
Charging state	No	Yes
GPS-only polling	No	Yes, via <code>GPS_ONLY</code>

11 End-to-End Data Flow

In standard mode, the end-to-end path is:

Pi Zero turns on L2 sensing circuit → Pi Zero sends `MEASURE 1600` → Pico measures frequency → Pico returns `DATA lat lon freq` → Pi Zero corrects frequency with IMU deltas → Pi Zero converts corrected frequency to nT → Pi Zero appends CSV row.

In boat mode, the measurement path is the same while the boat is in run mode. When the boat enters charging mode, the path changes to:

Boat sends charging command → Pi Zero holds L1/L2 low → Pi Zero sends `GPS_ONLY` periodically → Pico returns `GPS lat lon` → Pi Zero waits for a run/resume command.

12 Intermediate Tests

Several smaller scripts were used while bringing up the Pico, GPS receiver, and Pi Zero UART link. These are not the main operating programs, but they are worth keeping in the handoff because they isolate hardware and protocol problems more quickly than the full magnetometer cycle.

File	Why it matters
SP26_HookLineandTowfish_test_pico_MattSchutt.py	Best Pico-side integration test. It listens for the s <duration_ms> command as the real firmware and synthetic DATA <lat> <lon> <freq> rows. This v Zero/Pico UART protocol, newline framing, comman sponse parsing, and controller timing without needing quency input to work.
SP26_HookLineandTowfish_pingpong_MattSchutt.py	Useful intermediate communications test. It dumps r while also answering PING commands with PONG. This to confirm that the Pico can service the GPS UART a UART in the same loop before moving to full measur
SP26_HookLineandTowfish_timing_MattSchutt.py	Small GPS-only monitor for the GT-U7. It prints rec sentences and prints NO SIGNAL after a timeout. This GPS wiring/fix issues from the rest of the magnetom
SP26_HookLineandTowfish_bink_MattSchutt.py	Minimal Pico power and MicroPython sanity check. the onboard LED, but that makes it useful when deb flashing, or whether the Pico is running code at all.

13 Contact

For questions about this software handoff, da big software guy can be reached by email at

`mattschutt050@gmail.com`.

Good luck with the project.